PROJECT OVERVIEW

Our project will deliver an interactive graphical calculator as a single Verilog design on the Basys3 FPGA and a 96 x 64 OLED display. From a unified main menu, users will enter signed values with the board switches and select either a conventional calculator or an interactive graphing mode. The calculator will support addition, subtraction, multiplication and division, together with invalid-input detection, overflow handling and a short result history. Graph Mode will support one or two linear or quadratic functions entered as signed coefficients. It will render coloured curves on a scaled coordinate grid and, for two linear functions, calculate and mark their intersection while identifying parallel, identical, invalid and off-screen cases. To make the system more exploratory, the graph view also provides zoom, pan and a movable cursor with coordinate readout. Debounced controls, on-screen prompts, clear error messages and modular interfaces make the calculator reliable, easy to demonstrate and straightforward to extend with features such as graph tracing, automatic view fitting, saved function presets and additional mathematical operations.

GROUP MEMBER 1: MILLA GLÄNNFJORD

MEMBER RESPONSIBILITY

The main responsibility of this group member is to develop a module that reads switches and convert their states into binary numbers, including support for signed values. This group member will also write one module for detecting invalid user inputs and one potential overflow, and implementing a calculation history feature that stores and displays the three most recent calculation results. The modules will be designed so that the other group members can use it for both the calculator and graphing functions.

TIMELINE

Week 7: complete the practice task provided in the project manual to gain a better understanding of the OLED, Verilog programming and the different components needed for the project.

Week 8: develop the module responsible for reading the switches and interpreting their states as binary numbers. Ensure that the module can handle both positive and negative values and that it can be integrated with the other group members modules.

Week 9: implement additional functionality to detect invalid user inputs and potential overflow during calculations. Develop a history module that stores the three most recent calculation results.

Week 10: have a group meeting to review the progress of the project and to decide how to divide the remaining graphical components between the group members. Discuss how the different modules will be connected and integrated into the complete system.

Week 11: develop and implement my assigned graphical component for the OLED display. Test the graphical functionality and make necessary adjustments to ensure that it works correctly together with the other modules.

Week 12: have a final group meeting to integrate all the modules and complete the project. Perform final testing, identify and fix any remaining issues, and ensure that the system meets the project requirements.

GROUP MEMBER 2: HANNA WAHLGREN

MEMBER RESPONSIBILITY

The main responsibility is to develop and implement the calculator module, supporting the four basic arithmetic operations: addition, subtraction, multiplication and division. The calculator will use the FPGA board's buttons to select operations and switches to enter signed binary numbers. Another responsibility is to develop the graphical representation of the calculator on the OLED display, including digits, mathematical symbols, input values and calculation results. Furthermore, it will also develop the main menu logic in the top module, allowing users to navigate between the calculator and graphing functions, select a program and return to the main menu.

TIMELINE

Week 7: Completed the practice tasks in the project manual to gain a better understanding of the OLED display and overall verilog programming required for the project.

Week 8: Started developing the calculator-module and implemented the basic arithmetic operations using a state-based approach. Created a function for drawing digits (0-9) on the OLED display and began implementing the graphical representation of input values, mathematical operators and calculation results.

Week 9: Developed the main menu-logic in top-module, allowing users to navigate between the calculator and graphing programs using the buttons. Implemented functionality to select a program and return to the main menu. Worked on integrating Milla's modules for signed number input, invalid calculations, overflow detection and calculation history into the calculator module.

Week 10: Test the calculator module and its integration with the other modules. Improve the OLED graphics, including negative numbers, results and error messages. Participate in the group meeting to divide the remaining graphical tasks.

Week 11: Implement the assigned graphical components, including the main menu interface. Improve the calculator's graphical layout and test navigation between the menu and the different programs.

Week 12: Integrate the calculator and menu with the other group members' modules. Perform final testing on the FPGA board, fix remaining issues and prepare the system for the final demonstration.

GROUP MEMBER 3: WANG QICHEN

MEMBER RESPONSIBILITY

Design and implement the graph-rendering component of our FPGA-based graphical calculator. The renderer will convert the OLED pixel_index into coordinates for the 96 x 64 display and draw a readable graph area with axes, grid lines and labels. It will display two signed linear functions, y = k1x + m1 and y = k2x + m2, using different colours and continuous line-drawing logic. It will also show the calculated intersection with a visible marker and display clear messages for parallel, identical, invalid or off-screen results. During coefficient entry, the OLED will show the current input stage and short instructions for the board controls.

TIMELINE

Week 7: Completed the OLED familiarisation tasks and studied RGB565 colours, pixel_index conversion and basic Verilog drawing techniques.

Week 8: Investigated coordinate scaling, signed graph values, line-rendering methods and the interface required between the graph controller and OLED renderer.

Week 9: Defined the renderer architecture, colour scheme, graph states, intersection display and exceptional-case overlays, and coordinated the required signals with the controller component.

Week 10: Implement and simulate the core axes, grid, two-line renderer, input-stage text and intersection marker; then integrate it with the existing OLED module.

Week 11: Add zoom, pan, graph clipping, automatic scale selection, a movable cursor with coordinate labels, and optional quadratic rendering if FPGA resources permit.

Week 12: Optimise resource usage, verify timing, test signed and extreme inputs on the Basys3, complete group integration and prepare a reliable demonstration.

GROUP MEMBER 4: OSKAR PLOGNER

MEMBER RESPONSIBILITY

The main responsibility is to develop the graph controller and mathematical calculations for the graphing mode. This includes managing user inputs through a finite state machine (FSM), allowing users to select one or two graphs, choose function types and enter signed coefficients. Another major responsibility is calculating intersections between linear functions using fixed-point arithmetic, including handling parallel or identical lines, division by zero and overflow. The controller will communicate with the graph renderer by providing coefficients, calculation results and control signals. Additional features may include zoom, pan and cursor controls.

TIMELINE

Week 7: Completed the OLED familiarisation tasks and discussed possible project functionalities and responsibilities within the group.

Week 8: Planned the graphing functionality, including coefficient input, graph selection, intersection calculations and communication between the controller and renderer.

Week 9: Developed the initial graph_controller.v module using an FSM. Implemented selection of one or two graphs, linear/quadratic function selection, coefficient storage, reset functionality and signals indicating when the graphs are ready to be plotted. The module has not yet been tested or integrated.

Week 10: Simulate and test the graph controller with different input combinations. Begin integration with the signed-number input module and OLED renderer, adapting the interfaces between the modules. Implement intersection calculations for two linear functions, including handling parallel and identical lines.

Week 11: Implement error handling for invalid calculations, overflow and intersections outside the visible graph area. Continue testing and improving integration with the other modules. If time permits, implement additional controls for zoom, pan and cursor movement.

Week 12: Integrate the graph controller and intersection calculations with the other modules. Test the complete graphing functionality on the Basys3 board, fix remaining issues and help prepare the final demonstration.